The pipe operator

Day 5 - Friday, May 22, 2026

Today: Friday May 22

Before class

- Read *Data Computing* §3.6 (Click here).

Plan for today

- Quiz 1 (20 min)
- The pipe operator
- Chaining commands together
- In-class activity

Helpful links

- Syllabus · AI policy · Schedule · *Data Computing* §3.6

Quiz 1

Quiz 1 - instructions

Closed-book, no AI, no electronics

- Put away phones and laptops.
- You may use a pen or pencil.
- 20 minutes.

Topics covered:

- R as a calculator and basic syntax

- Data types and structures
- Objects and assignment
- Reading R code (functions, arguments, variables, constants)
- Packages: installing vs. loading
- Histograms in `ggplot2` (reading code, not writing from scratch)

When you finish, bring your quiz up to the front.

Quiz debrief

- We'll go over common stumbling points on Monday after I've graded them.

The pipe operator

A problem we've already seen

Remember this code from Wednesday?

```
sqrt(mean(c(1, 4, 9, 16, 25)))
```

```
[1] 3.316625
```

This should be read from the **inside-out**:

1. First `c(1, 4, 9, 16, 25)` makes a vector.
2. Then `mean(...)` of that vector is `11`.
3. Then `sqrt(11)` is $\approx$ `3.317`.

As the number of functions you nest grows, the harder and harder it gets to read.

A real example

Suppose we want to:

1. Take the `penguins` data,
2. Keep only rows where `body_mass_g` is not missing,
3. Group by `species`,
4. Compute the mean body mass for each species.

Written with nested function calls:

```
summarize(
  group_by(
    filter(penguins, !is.na(body_mass_g)),
    species
  ),
  avg_mass = mean(body_mass_g)
)
```

You have to read it from the **inside out**, and the arguments get separated from their functions. This is not ideal or intuitive for humans.

Enter the pipe

The **pipe operator** lets you read code left-to-right, top-to-bottom, which is the way you actually think about it.

```
penguins |>
  filter(!is.na(body_mass_g)) |>
  group_by(species) |>
  summarize(avg_mass = mean(body_mass_g))
```

Read this as:

> "Take `penguins`, **then** filter out missing values, **then** group by species, **then** summarize the average mass."

That's it. That's the whole idea.

What the pipe does

```
x |> f()
```

is **exactly the same** as

```
f(x)
```

The pipe takes whatever's on the **left** and inserts it as the **first argument** to the function on the **right**.

```r
# These do the same thing:
sqrt(16)
```

```
[1] 4
```

```r
16 |> sqrt()
```

```
[1] 4
```

With multiple arguments

```r
x |> f(arg2, arg3)
```

is the same as

```r
f(x, arg2, arg3)
```

The pipe always fills the **first** argument; the rest go inside the parentheses as usual.

```r
# Without the pipe:
round(3.14159, digits = 2)
```

```
[1] 3.14
```

```r
# With the pipe:
3.14159 |> round(digits = 2)
```

```
[1] 3.14
```

Chaining pipes

It is very useful to chain pipes together to do many steps in order.

```
c(1, 4, 9, 16, 25) |>
  mean() |>
  sqrt()
```

```
[1] 3.316625
```

Compare to the nested version:

```
sqrt(mean(c(1, 4, 9, 16, 25)))
```

```
[1] 3.316625
```

Here, we get the same answer. The piped version reads in the order you do the work.

Style: one pipe per line

> Convention
>
> Put each pipe step on its **own line**, with the `|>` at the **end** of the previous line. Indent the continuation by 2 spaces.

Good:

```
penguins |>
  filter(!is.na(body_mass_g)) |>
  group_by(species) |>
  summarize(avg_mass = mean(body_mass_g))
```

Not so good (works, but much harder to read):

```
penguins |> filter(!is.na(body_mass_g)) |> group_by(species) |> summarize(avg_mass = mean(bo
```

Two pipes?

|> vs. %>%

You will see **two** pipe operators in R code online:

Table 1: The two pipes in R.

Pipe	Where it comes from	When introduced
%>%	The `magrittr` package (and tidyverse)	2014
|>	Base R	2021

They do **almost the same thing**. There are slight differences, but for the purposes of this course, the distinction is not very significant.

Why two pipes?

- %>% came first. It's the original tidyverse pipe from the magrittr package.
- Base R added |> later because the pipe became so popular they wanted it built in.
- Note that the textbook (*Data Computing*) uses %>%; I'll mostly use |>.

> **Note**
>
> **You can use either one.** But try to be consistent within a single document.

Typing the pipe quickly

You don't have to type |> character-by-character. RStudio has a shortcut:

- **Mac:** `Cmd + Shift + M`
- **Windows/Linux:** `Ctrl + Shift + M`

> **Tip**
>
> By default, this shortcut inserts %>%. To make it insert |> instead:
> `Tools > Global Options... > Code > Use native pipe operator`

Reading piped code

Translating between styles

You'll see code in both styles all over the internet. Practice translating in your head.

Nested:

```
head(arrange(filter(penguins, species == "Gentoo"), body_mass_g), 5)
```

Piped:

```
penguins |>
  filter(species == "Gentoo") |>
  arrange(body_mass_g) |>
  head(5)
```

This gives the same result, and is **much** easier to read the piped version.

Limitations of the pipe

⚠ The pipe doesn't run things in parallel

It just rewrites your code. Each step still runs one at a time, in order.

⚠ The pipe is NOT the same as + in ggplot2

- `dplyr` and other data verbs: use `|>` or `%>%`.
- `ggplot2` layers: use `+` (it was developed before the pipe).

```
# Correct:
penguins |>
  filter(!is.na(body_mass_g)) |>
  ggplot(aes(x = body_mass_g)) +
    geom_histogram()
```

A peek at what's coming

You'll see the pipe very often from here on out. Next time we start the **dplyr** verbs:

- `filter()` - pick rows
- `select()` - pick columns
- `arrange()` - sort
- `mutate()` - make new columns
- `summarize()` - collapse to a single number
- `group_by()` - split into groups before summarizing

Every one of these is designed to be chained together with the pipe.

In-class activity

Activity: pipe practice (~ 15 min)

- Click here for the activity

To turn in: Save your script. We'll check at the end of class.

Activity debrief

Common things to watch for:

- **The pipe goes at the end of the line**, not the start. `x |>` then a new line - not `\n |> f()`.
- **The first argument is filled automatically.** Don't write `penguins |> filter(penguins, ...)`: drop the first `penguins`.
- **`+` vs. `|>` again.** dplyr verbs chain with `|>`; ggplot layers chain with `+`.
- **Parentheses still matter.** `f()` not `f`. The pipe doesn't change that.

Wrap-up: what we covered today

- The pipe operator (`|>` or `%>%`) takes the value on the left and feeds it as the first argument to the function on the right.
- Pipes let you read code top-to-bottom in the order you'd actually do the work.
- Each step gets its own line; the pipe lives at the **end** of the previous line.
- `|>` and `%>%` are nearly interchangeable for our purposes.
- dplyr verbs chain with `|>`; ggplot layers chain with `+`.

Wrap-up & looking ahead

Tuesday: Data frames · intro to data graphics.

Upcoming Deadlines

- **Today, 11:59 p.m.** - HW 1: Basic R & Plots
- **Mon 5/25, before class** - DC §5.1-5.6

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *Data Computing*